Project 2 – Acme Analytics Linux Platform

Day 5 · Phase 2 complete, Phase 3 started: snapshot rollback and job control

Muhammad Bilal Javed · 20 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	2 – Storage, sharing and recovery · 3 – Automation
Hosts worked on	gw01 (access test), stor01 (LVM snapshot, NFS server), app01 (NFS client)
Status	Phase 2 ✅ – Steps 2.1–2.4 done; VMware snapshots `30-storage-*` · Phase 3 🟡 – Step 3.1 job control ✅
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 4.

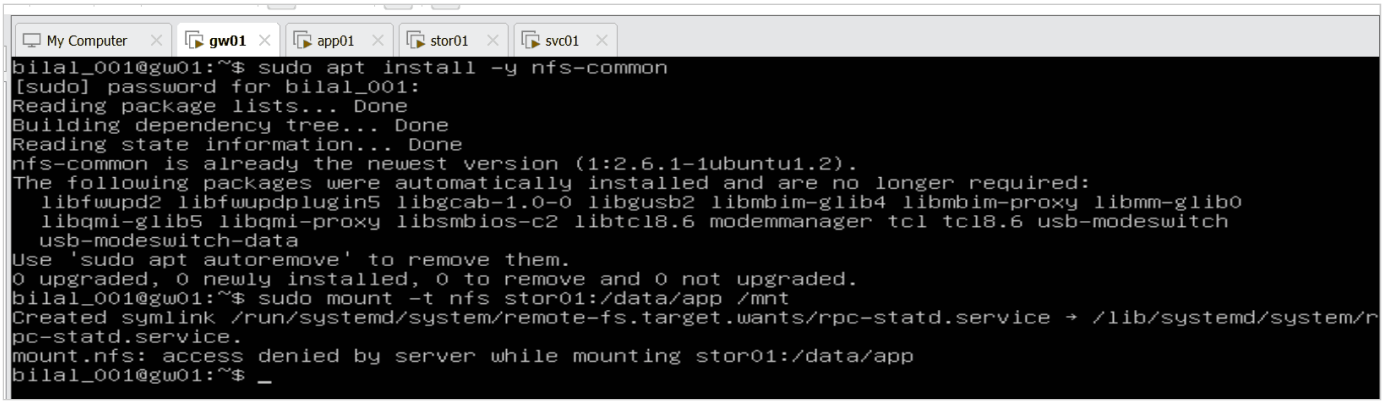
---

1. Break test – reachable is not allowed (Step 2.2)

💡 **In plain words:** gw01 has stor01 in `/etc/hosts` and can ping it – but it is not listed in stor01's `/etc/exports`. Can it use the share anyway?

```
# on gw01
sudo apt install -y nfs-common                  # NFS client tools (already installed)
sudo mount -t nfs stor01:/data/app /mnt         # /mnt = standard empty folder for temporary mounts
```

📸 **Evidence – `access denied by server`:**



access denied

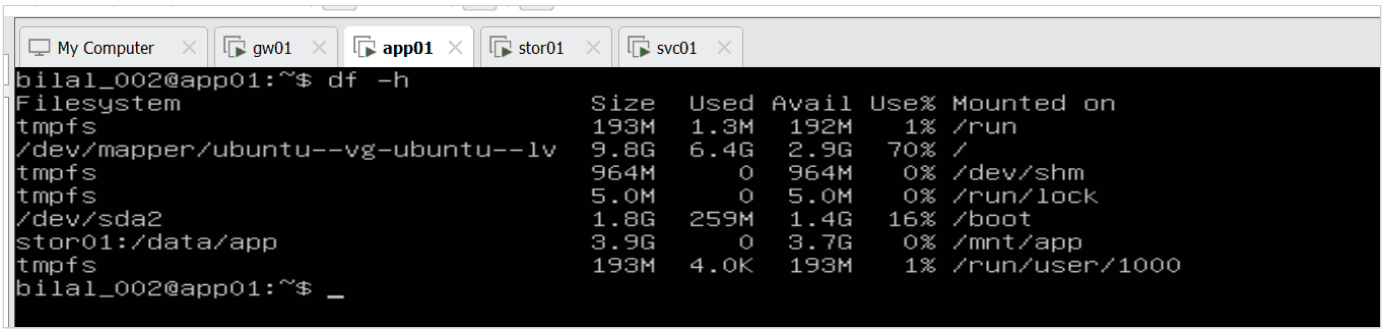
Lesson: being able to reach a server is not permission to use its shares. Only the IPs listed in `/etc/exports` (here only app01, 192.168.60.10) may mount `/data/app`.

2. Where the app's data really lives

```
app01: /mnt/app  ═ NFS ═▶  stor01: /data/app  ─inside─▶  /data   ─mounted from─▶  lvdata (logical volume)
                                                          /backups ─mounted from─▶  lvbackups
```

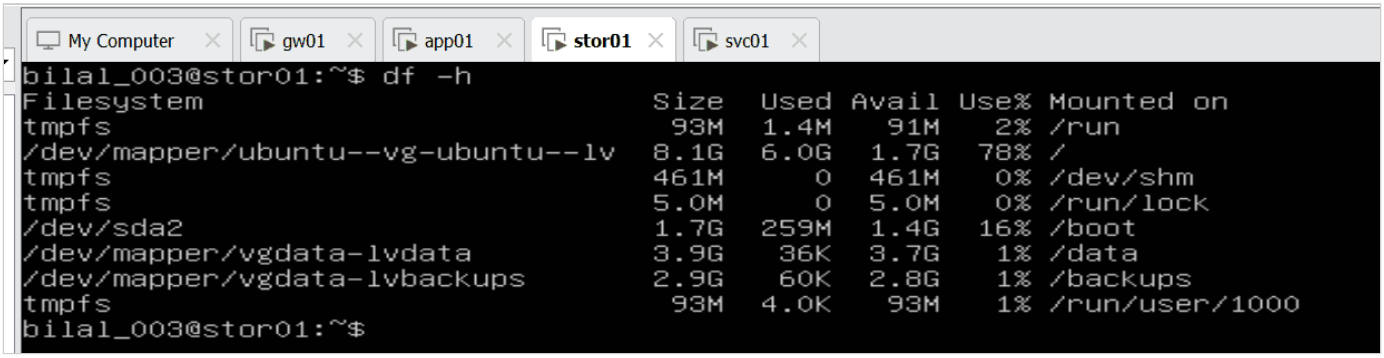
- `/mnt/app` is a folder on **app01**; `/data/app` is a folder on **stor01**. Over NFS, everything done in `/mnt/app` really happens in `/data/app`.
- On stor01 the logical volume **`lvdata`** is mounted on `/data` – so every file in `/data/app` is stored on `lvdata`. **The work happens on `lvdata`; the backups are on another volume, `lvbackups`.**

📸 **Evidence – app01: `stor01:/data/app` on `/mnt/app`:**



app01 df

📸 **Evidence – stor01: `lvdata` on `/data`, `lvbackups` on `/backups`:**



stor01 df

3. LVM snapshot – the undo button (Step 2.4)

💡 **The story:** a risky change is coming to `/data` (an update, a cleanup). A mistake made from app01 lands directly on `lvdata`, and the rsync mirror – which keeps only the current state – would copy it at its next run. The dated versions still hold good files, but restoring file by file is slow. So **before** the change: take a snapshot – a frozen state of the whole volume – and roll back in seconds if it goes wrong.

3.1 Take the snapshot, make the “mistakes”

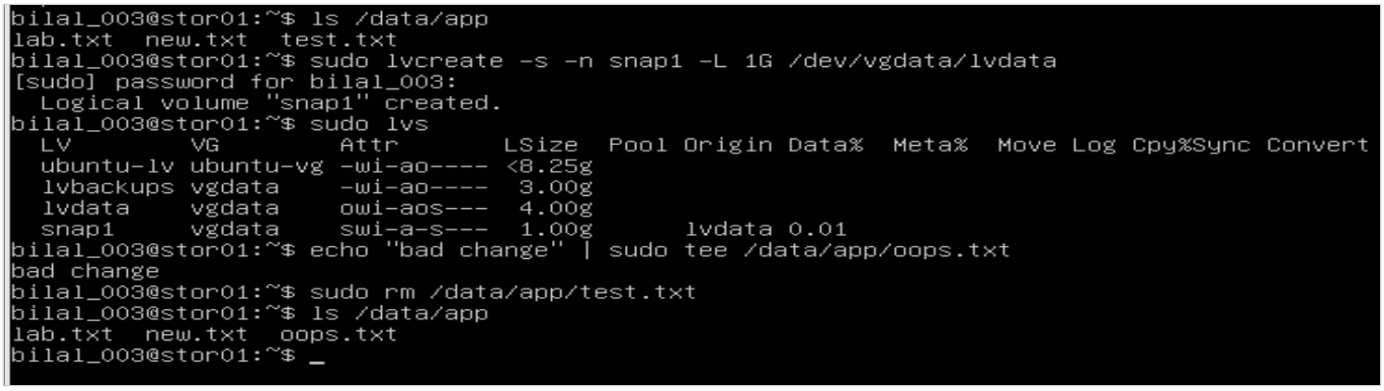
```
# on app01 — release the share first (see the issue below)
sudo umount /mnt/app

# on stor01
ls /data/app                                          # lab.txt new.txt test.txt
sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata    # -s snapshot of lvdata; 1 GB room for changes
sudo lvs
echo "bad change" | sudo tee /data/app/oops.txt       # mistake 1
sudo rm /data/app/test.txt                            # mistake 2
ls /data/app                                          # lab.txt new.txt oops.txt
```

`lvs` shows	Meaning
`lvdata  owi-aos---`	**o** = origin: a snapshot is watching this volume
`snap1  swi-a-s---  1.00g  Origin lvdata`	**s** = snapshot of lvdata, 1 GB room for changes
`Data% 0.01`	How full the snapshot’s room is – at 100 % it becomes invalid

The mistakes were made on stor01 for simplicity; from app01 through `/mnt/app` the effect is identical – it is the same file on `lvdata`, and the snapshot protects the volume, whoever made the change.

📸 **Evidence:**

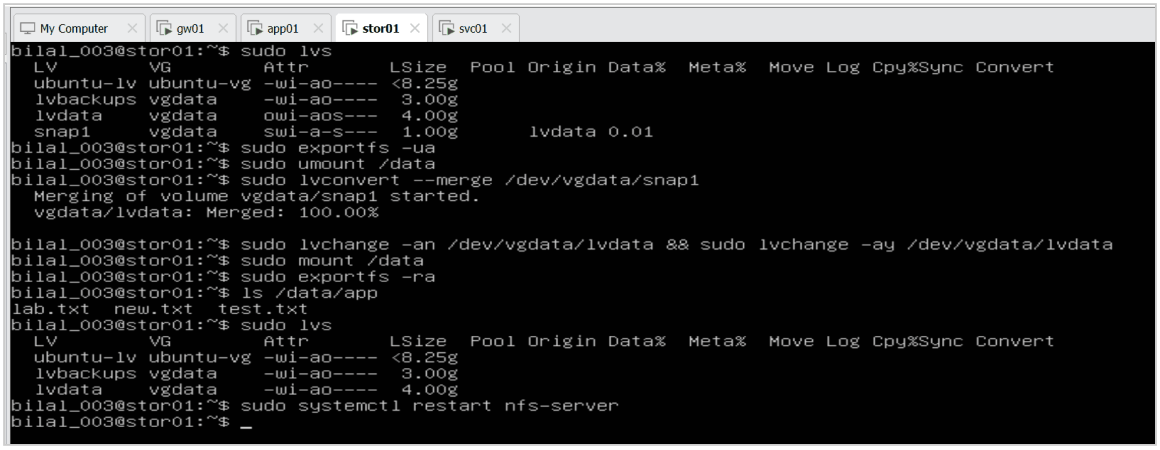


snapshot taken

3.2 Roll back

```
# on stor01
sudo exportfs -ua                                     # stop sharing /data/app
sudo umount /data                                     # nobody may use the volume during the merge
sudo lvconvert --merge /dev/vgdata/snap1              # back to the frozen state
sudo lvchange -an /dev/vgdata/lvdata && sudo lvchange -ay /dev/vgdata/lvdata   # deactivate / activate
sudo mount /data
sudo exportfs -ra                                     # share again
ls /data/app                                          # lab.txt new.txt test.txt — oops.txt gone
sudo lvs                                              # snap1 gone — merged into lvdata
```

📸 **Evidence – `Merged: 100.00%`; test.txt back, oops.txt gone; snap1 gone:**

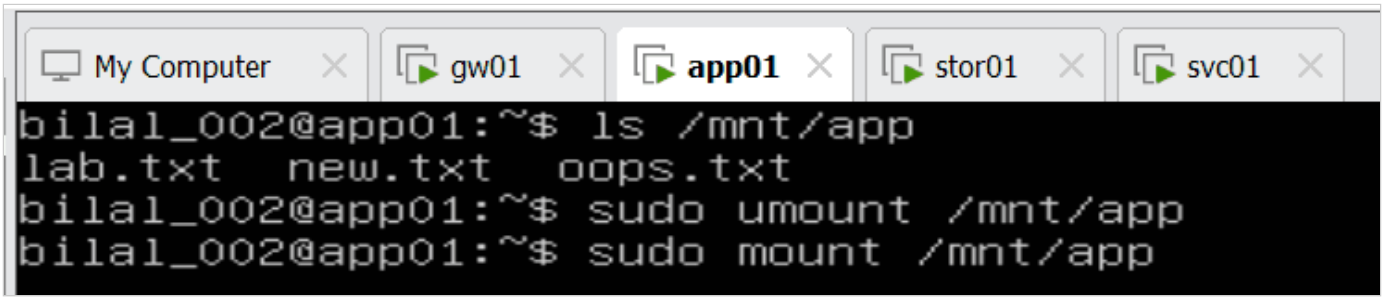


rollback

3.3 Issue – app01 hangs after the rollback (stale NFS mount)

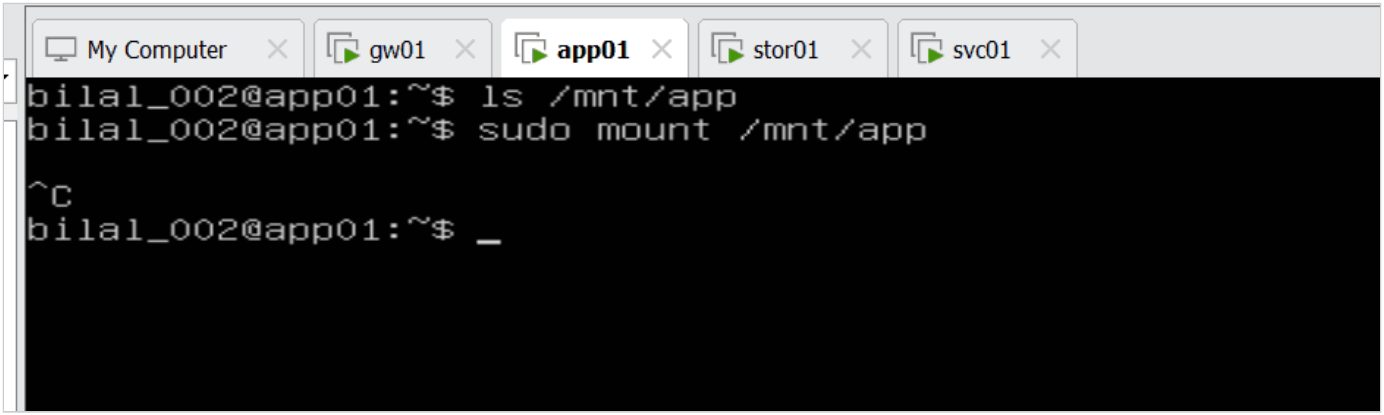
Seen	After the rollback, `mount /mnt/app` on app01 hung and had to be stopped with Ctrl+C.
Cause	app01 still had the share mounted **during** the rollback – `ls /mnt/app` still showed `oops.txt`; it was unmounted only afterwards. stor01 pulled the volume away under a connected client, which was left with an old, invalid connection (a **stale** mount).
Fix	stor01: `sudo systemctl restart nfs-server` · app01: `sudo umount -f -l /mnt/app` (force, lazy), `showmount -e stor01`, `sudo mount -a`
Lesson	Before rolling back a volume shared over NFS, **unmount every client first** – the client lets go, then the server.

📸 **Evidence – app01 still mounted during the rollback (oops.txt visible):**



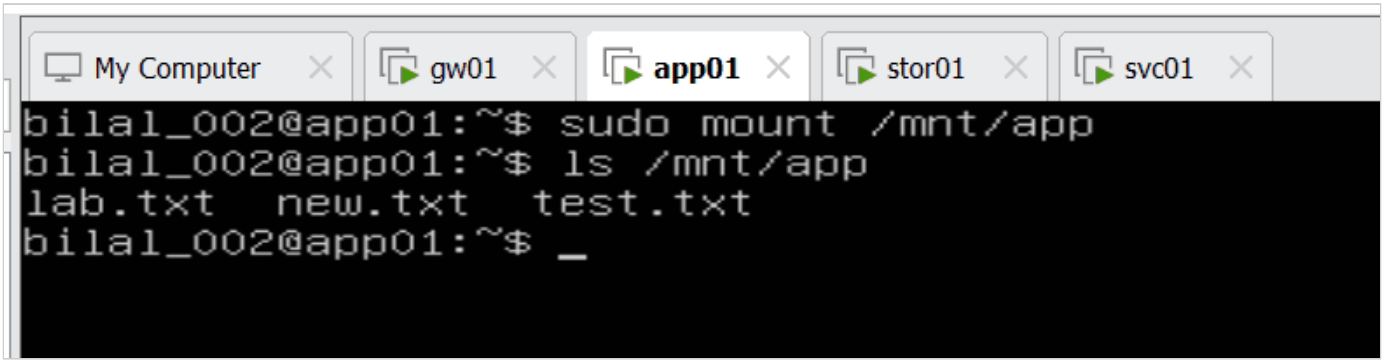
still mounted

📸 **Evidence – the mount hangs:**



hang

📸 **Evidence – after the fix, app01 sees the rolled-back files:**



fixed

4. Snapshot vs backup

	LVM snapshot	Backup (versions in `/backups`)
What	A **frozen state of the whole volume** at the moment it was taken	**Separate copies** of the files
How	Keeps only the old version of blocks that change afterwards (copy-on-write), in its small room	Full files (unchanged ones hard-linked)
Where	**Same disks** as `lvdata`	Separate volume `lvbackups` – ideally another server
Restore	The **whole volume at once**, in seconds	Files **one by one**, from any day
Survives a reboot	Yes	Yes
Lost when	Disks die, its room fills to 100 %, merged or removed	Deleted, or the disks die – hence copies off-site

Snapshot = quick undo, short-term, same disks. Backup = recovery, long-term, separate volume.

5. VMware snapshots

VM	Snapshot	Description
gw01	`30-storage-gw01`	End of Phase 2; NFS client tools for the access test
app01	`30-storage-app01`	NFS client: stor01:/data/app on /mnt/app (fstab, `_netdev,nofail`)
stor01	`30-storage-stor01`	LVM vgdata (lvdata /data 4 GB, lvbackups / backups 3 GB), NFS server, tar/rsync backups v1–v3
svc01	`30-storage-svc01`	GLPI; restore ticket closed

Key understanding (in my own words)

> On app01, `/mnt/app` is a folder. On stor01, `/data/app` is a folder. They are connected over NFS, so everything done in `/mnt/app` on app01 really happens in `/data/app` on stor01. On stor01, the logical volume `lvdata` is mounted on `/data`, so every file in `/data`, including `/data/app`, is stored on `lvdata`. The work happens on `lvdata`; the backups are stored on another logical volume, `lvbackups`, mounted on `/backups`.
>
> If someone on app01 makes a wrong or risky change in `/mnt/app`, the mistake lands directly on `lvdata`. The rsync mirror only keeps the current state, so at its next run it would copy the mistake too. A backup is a set of separate copies of the files on `lvbackups`: from it I can restore files one by one, from any day, but that is slow for a big mistake.
>
> So before a risky change I take an **LVM snapshot**: a frozen state of the whole `lvdata` volume, taken at the moment I create it. If the change goes wrong, I roll back the whole volume at once. To roll back, nobody may be using the volume: first app01 unmounts `/mnt/app`, then stor01 stops sharing (`exportfs -ua`) and unmounts `/data`. I learned this the hard way – app01 was still mounted during my rollback, was left with a stale mount and hung, until I restarted the NFS server and force-unmounted the client.
>
> Both survive a reboot, but a snapshot lives on the same disks and becomes invalid if its room fills up: **snapshot = quick undo, short-term, same disks; backup = recovery, long-term, separate volume.** And gw01 showed that being able to reach a server is not permission to use its shares.

6. Phase 3 starts – automation (Step 3.1: job control)

💡 **The story:** so far every command ran by hand, and long commands blocked the terminal. Acme's customers upload large files from **app01**; through NFS they land in `/data/app/uploads` on **stor01**, where they must be backed up. Long jobs like these should **not block my terminal**, **not slow down the app**, and **not die when my connection drops**.

```
app01: /mnt/app/uploads  ═ NFS ═▶  stor01: /data/app/uploads (lvdata)  ─backup─▶  /backups (lvbackups)
       "customers upload"                                                  nohup tar from an SSH session
```

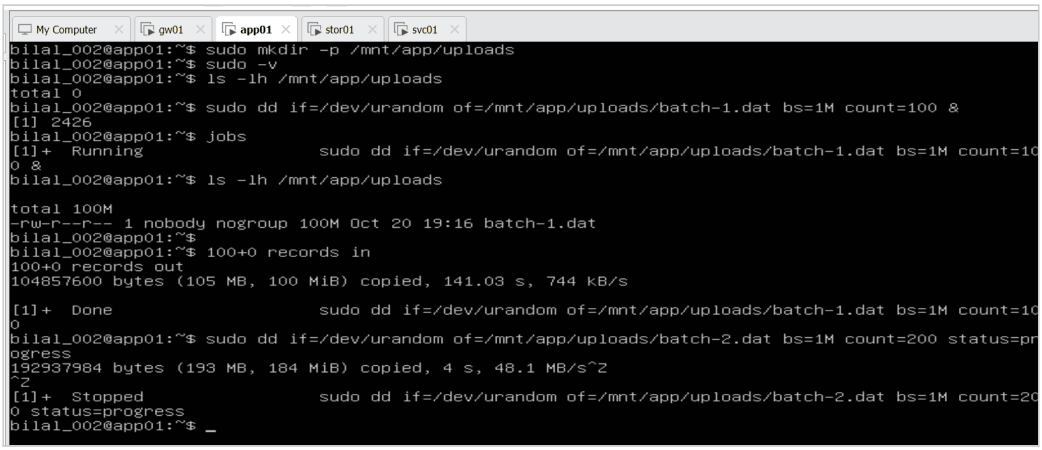
6.1 Customer uploads on app01 – foreground, background, pause, priority, kill

```
        # app01
        sudo rm -rf /mnt/app/uploads && sudo mkdir -p /mnt/app/uploads   # clean start (really /data/app/uploads on
stor01)
        sudo -v                                                          # cache the password — background sudo jobs
cannot ask for it

        # batch-1: in the background
        sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=100 &
        jobs                                                             # [1]+ Running — the prompt is free
        ls -lh /mnt/app/uploads                                          # keep working while it uploads

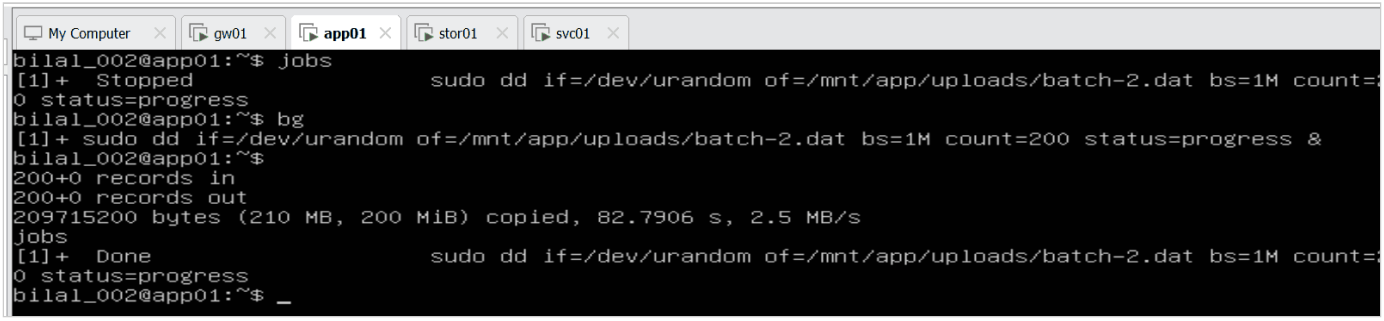
        # batch-2: forgot the & — pause and send to the background
        sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-2.dat bs=1M count=200 status=progress
        #   Ctrl+Z → [1]+ Stopped
        jobs; bg; jobs                                                   # Stopped → Running → Done
```

📸 **Evidence – batch-1 in the background (`Running` → `Done`), batch-2 paused with Ctrl+Z (`Stopped`):**



bg and ctrl+z

📸 **Evidence – `bg`: it continues in the background and finishes:**



bg done

Issue – Ctrl+Z and Ctrl+C seemed to do nothing

Seen	During an upload to `/mnt/app`, `^Z ^C ^Z …` appeared on the screen but the job did not react – until, after a while, `[1]+ Stopped`.
Cause	dd was writing to a **network disk**. While a process waits for NFS to confirm a write it is in state **D – uninterruptible sleep** and cannot react to signals until that write finishes.
Check	`ps -o pid,stat,cmd -C dd` – STAT `D` = waiting on the disk
Lesson	A slow or hanging NFS server can freeze programs on the client – the same root as yesterday's stale mount. dd's speed (575 kB/s) also counts the paused minutes; the real speed was about 9 MB/s.

📸 **Evidence:**

```
bilal_002@app01:~$ sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=100
^Z


^C



^C^Z^Z^Z^Z^Z^Z



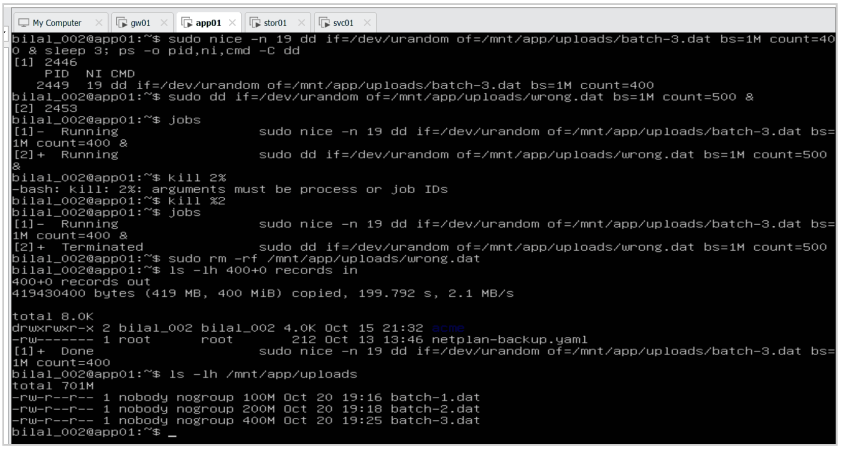
[1]+  Stopped                 sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=10
0
bilal_002@app01:~$ jobs
[1]+  Stopped                 sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=10
0
bilal_002@app01:~$ _
```

ctrl+z waits

```
        # batch-3: low priority — start dd and check its nice value in the same line
        sudo nice -n 19 dd if=/dev/urandom of=/mnt/app/uploads/batch-3.dat bs=1M count=400 & sleep 3; ps -o pid,ni,cmd -
C dd

        # wrong.dat: started by mistake — stop only that job
        sudo dd if=/dev/urandom of=/mnt/app/uploads/wrong.dat bs=1M count=500 &
        jobs                                # [1] batch-3 Running, [2] wrong.dat Running
        kill %2                             # [2] Terminated — batch-3 keeps running
        sudo rm -f /mnt/app/uploads/wrong.dat
        ls -lh /mnt/app/uploads             # batch-1 100M, batch-2 200M, batch-3 400M
```

📸 **Evidence – `NI 19`; two jobs at once, only `%2` killed; the three batches, owned by `nobody` (root_squash):**



nice kill result

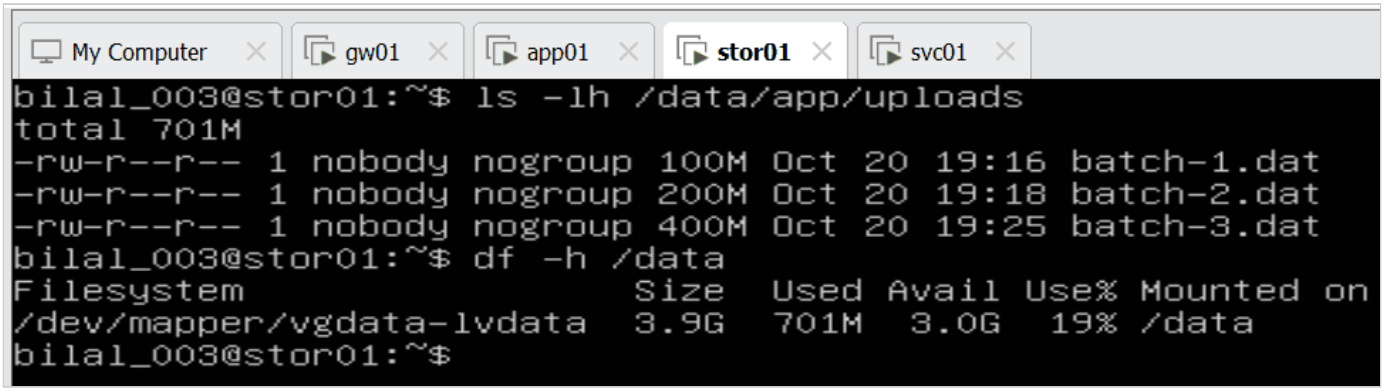
Tool	Meaning	Real use
&	Start in the background	A long upload or backup without blocking the terminal
Ctrl+Z → bg / fg	Pause, continue in background / foreground	Rescue a job started without &
jobs	List my jobs with numbers	
kill %2 / kill PID	Stop by job number (same terminal) / by process ID (anywhere)	A job started by mistake
nice -n 19 / renice -n 19 -p PID	Priority -20 (most important) … 19 (politest), at start / while running	A daytime backup that does not slow the app
nohup … &	Keep running after logout	A long job over an SSH connection that may drop

&/fg/bg decide **where** a job runs (does it block the terminal); nice/renice decide **how important** it is when the CPU is busy – all jobs still run at the same time. For disk-heavy jobs `ionice -c3` is the disk equivalent.

6.2 On stor01 – the uploads arrived; a backup that survives a logout

```
        # stor01
        ls -lh /data/app/uploads          # the three batches, 701 MB
        df -h /data                       # 19 % used
```

📸 **Evidence:**

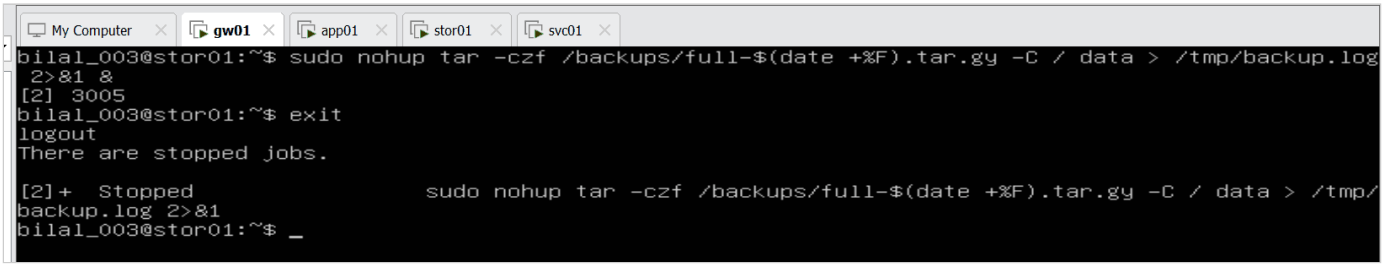


uploads arrived

Issue – the background backup went “Stopped”

Seen	The second `sudo nohup tar … &` showed `[2]+ Stopped`, and `exit` answered *There are stopped jobs* instead of logging out.
Cause	sudo wanted a password. A background job **cannot read from the keyboard**, so Linux paused it. Bash refuses the first `exit` while a job is stopped.
Lesson	Run `sudo -v` in the foreground **before** a background sudo job, and check `jobs` before `exit`. (The archive name also had a typo, `.tar.gy` – removed.)

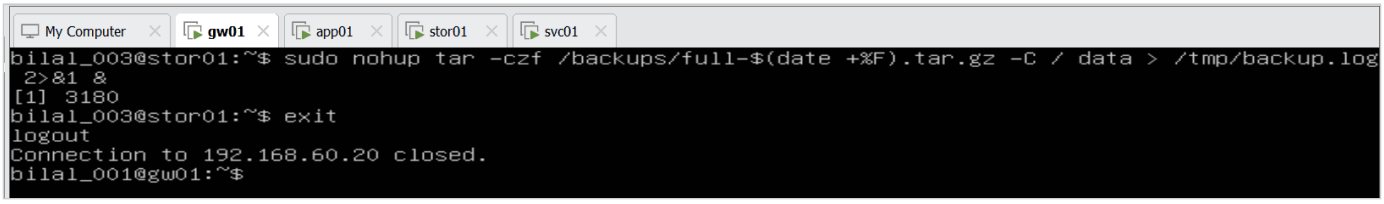
📸 **Evidence:**



stopped

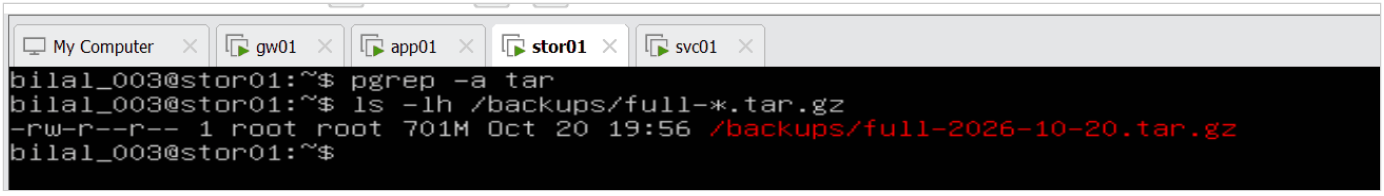
```
# gw01 — plays "my laptop"
ssh stor01
sudo -v
sudo nohup tar -czf /backups/full-$(date +%F).tar.gz -C / data > /tmp/backup.log 2>&1 &
exit                                    # really logged out: "Connection to 192.168.60.20 closed."
ssh stor01
pgrep -a tar                            # nothing — finished
ls -lh /backups/full-*.tar.gz           # 701M, finished at 19:56 — after I had left
sudo rm -f /backups/full-*.tar.gz       # free the space again
```

📸 **Evidence – started, logged out:**



nohup logout

📸 **Evidence – the archive was finished while I was gone:**



finished after logout

The archive is as big as the data (701 MB): random files – like real photos, videos and zipped uploads – hardly compress. Text and logs shrink a lot. That matters when planning backup space.

Key understanding (in my own words)

Acme's customers upload files from app01 through `/mnt/app`, which NFS stores in `/data/app/uploads` on stor01. Long jobs like these uploads and backups should not block my terminal or slow down the app.

Foreground and background: any command can run in the foreground or the background. In the foreground, the terminal is busy until the command finishes. With `&` it runs in the background, and I get the prompt back at once. If I forget the `&`, `Ctrl+Z` pauses the job and `bg` lets it continue in the background; `fg` brings it back. `jobs` lists my jobs with their numbers. A wrong job is stopped with `kill %number` (job number, same terminal) or `kill PID` (process ID, from anywhere). On a network disk (NFS), a job waiting for a write reacts to Ctrl+Z or kill only after that write finishes.

Priority: `nice` and `renice` set a job's priority from -20 (most important) to 19 (politest). All jobs run at the same time, but when the CPU is busy, lower numbers get more CPU time. `nice` sets it at the start; `renice` changes it while the job runs. That is how a backup can run during the day without slowing down the app.

Surviving a logout: a job started from an SSH session is normally killed when the connection closes. With `nohup … &` it keeps running on the server after I log out. I started a 700 MB backup on stor01 from gw01, logged out, came back, and the archive had been finished while I was gone. Two lessons: run `sudo -v` first, because a background job cannot answer a password prompt (it becomes "Stopped"); and check `jobs` before `exit`, because Bash will not log out while a job is stopped. The archive was as big as the data (701 MB), because random files like customer uploads hardly compress.

7. Next steps

1. **Step 3.2:** disk-full alert as a systemd **service + timer** on stor01 – stor01 checks `/data` itself every 5 minutes.
2. Steps 3.3–3.5: socket-activated service, custom target, cron vs timer – and the **nightly backup** running by itself.
3. Phase 2 tickets in GLPI: LVM build, grow /data, fstab UUID typo, NFS share, stale NFS mount after rollback.